
LAPORAN PRAKTIKUM PEMROGRAMAN BERORIENTASI OBJEK

MODUL 8 - POLIMORFISME

(Polymorphism)



NAMA : MUHAMAD KRISNA YODI PRATAMA

NIM : 2 5 1 1 0 4 4 1 0 0 8 3

PERIODE : SEMESTER GENAP 2025/2026

**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ISLAM BALITAR 2026**

8.1 Polimorfisme pada Constructor (Method Overloading)

Constructor overloading itu kalau satu class punya lebih dari satu constructor tapi parameternya beda-beda. Di class Music ini ada 3 constructor: tanpa parameter, 2 parameter, dan 3 parameter. Jadi kita bisa buat objek Music dengan cara yang beda-beda.

```
Music.java
1  public class Music {
2      private String judul;
3      private String artis;
4      private int durasi;
5
6      public Music() {
7          this.judul = "Unknown";
8          this.artis = "Unknown";
9          this.durasi = 0;
10     }
11
12     public Music(String judul, String artis) {
13         this.judul = judul;
14         this.artis = artis;
15         this.durasi = 0;
16     }
17
18     public Music(String judul, String artis, int durasi) {
19         this.judul = judul;
20         this.artis = artis;
21         this.durasi = durasi;
22     }
23
24     public void tampilkanInfo() {
25         System.out.println("Judul : " + judul);
26         System.out.println("Artis : " + artis);
27         System.out.println("Durasi : " + durasi + " detik");
28     }
29 }
```

Constructor pertama ngasih nilai default, kedua isi judul sama artis aja, ketiga isi semua. Method `tampilkanInfo()` buat nampilin datanya.

MainMusic.java

```
1 public class MainMusic {
2     public static void main(String[] args) {
3         System.out.println("---- Music 1 ----");
4         Music music1 = new Music();
5         music1.tampilkanInfo();
6
7         System.out.println("---- Music 2 ----");
8         Music music2 = new Music("Halo", "Adele");
9         music2.tampilkanInfo();
10
11        System.out.println("---- Music 3 ----");
12        Music music3 = new Music("Bohemian Rhapsody", "Queen", 354);
13        music3.tampilkanInfo();
14    }
15 }
```

Output:

```
---- Music 1 -----
Judul : Unknown
Artis : Unknown
Durasi : 0 detik
---- Music 2 -----
Judul : Halo
Artis : Adele
Durasi : 0 detik
---- Music 3 -----
Judul : Bohemian Rhapsody
Artis : Queen
Durasi : 354 detik
```

Music 1 pake constructor tanpa parameter jadi semua default. Music 2 pake 2 parameter, durasi masih 0. Music 3 pake 3 parameter jadi lengkap. Ini bukti kalau satu class bisa punya banyak bentuk constructor.

8.2 Method Overriding

Method overriding itu subclass nulis ulang method dari superclass. Tanda `@Override` nunjukin kalau method itu benar-bener di-override. Di contoh ini, class `Pembayaran` punya method `prosesBayar()` dan `getMetode()`, lalu subclass `GoPay`, `TransferBank`, dan `QRIS` nulis ulang methodnya masing-masing.

Pembayaran.java

```

1  public class Pembayaran {
2      protected double nominal;
3
4      public Pembayaran(double nominal) {
5          this.nominal = nominal;
6      }
7
8      public void prosesBayar() {
9          System.out.println("Memproses pembayaran Rp " + nominal);
10     }
11
12     public String getMetode() {
13         return "Pembayaran Umum";
14     }
15 }

```

GoPay.java

```

1  public class GoPay extends Pembayaran {
2      public GoPay(double nominal) {
3          super(nominal);
4      }
5
6      @Override
7      public void prosesBayar() {
8          System.out.println("[GoPay] Membayar Rp " + nominal
9              + " via dompet digital");
10     }
11
12     @Override
13     public String getMetode() {
14         return "GoPay";
15     }
16 }

```

TransferBank.java

```

1  public class TransferBank extends Pembayaran {
2      private String namaBank;
3

```

```

4      public TransferBank(double nominal, String namaBank) {
5          super(nominal);
6          this.namaBank = namaBank;
7      }
8
9      @Override
10     public void prosesBayar() {
11         System.out.println "[" + namaBank + "] Transfer Rp "
12         + nominal + " via rekening Bank");
13     }
14
15     @Override
16     public String getMetode() {
17         return "Transfer " + namaBank;
18     }
19 }

```

QRIS.java

```

1      public class QRIS extends Pembayaran {
2          public QRIS(double nominal) {
3              super(nominal);
4          }
5
6          @Override
7          public void prosesBayar() {
8              System.out.println("[QRIS] Scan barcode - membayar Rp "
9              + nominal);
10         }
11
12         @Override
13         public String getMetode() {
14             return "QRIS";
15         }
16     }

```

MainPembayaran.java

```

1      public class MainPembayaran {
2          public static void main(String[] args) {

```

```

3     Pembayaran[] transaksi = {
4         new GoPay(75000),
5         new TransferBank(50000, "BCA"),
6         new QRIS(25000)
7     };
8     for (Pembayaran p : transaksi) {
9         p.prosesBayar();
10        System.out.println(" Metode : " + p.getMetode());
11        System.out.println("----");
12    }
13 }
14 }

```

Output:

```

[GoPay] Membayar Rp 75000.0 via dompet digital
Metode : GoPay
----
[BCA] Transfer Rp 50000.0 via rekening Bank
Metode : Transfer BCA
----
[QRIS] Scan barcode - membayar Rp 25000.0
Metode : QRIS
----

```

Semua objek disimpan di array Pembayaran, tapi method yang dijalankan tetap milik subclass. Ini karena dynamic binding, JVM mengecek tipe objek aslinya saat runtime.

8.3 Dynamic Binding (Late Binding / VMI)

Dynamic binding berarti JVM baru nentuin method mana yang dipanggil saat runtime, bukan saat compile. JVM liat tipe aktual objeknya, bukan tipe variabelnya. Ini juga disebut VMI (Virtual Method Invocation).

```

Hewan.java
1     class Hewan {
2         public void suara() {
3             System.out.println("...");
4         }
5     }
6
7     class Kucing extends Hewan {
8         @Override

```

```

9      public void suara() {
10         System.out.println("Meong");
11     }
12 }
13
14 class Anjing extends Hewan {
15     @Override
16     public void suara() {
17         System.out.println("Guk guk");
18     }
19 }
20
21 class Burung extends Hewan {
22     @Override
23     public void suara() {
24         System.out.println("Cuit cuit");
25     }
26 }

```

ContohDynamicBinding.java

```

1      public class ContohDynamicBinding {
2          public static void main(String[] args) {
3              Hewan h1 = new Kucing();
4              Hewan h2 = new Anjing();
5              Hewan h3 = new Burung();
6
7              h1.suara();
8              h2.suara();
9              h3.suara();
10         }
11     }

```

Output:

```

Meong
Guk guk
Cuit cuit

```

Pertanyaan Analisa

1. Mengapa method suara() yang dipanggil milik subclass meskipun h1, h2, h3 dideklarasikan dari class Hewan?

Karena dynamic binding (VMI). JVM saat runtime ngecek tipe objek aslinya, bukan tipe variabelnya. h1 isinya new Kucing(), jadi suara() milik Kucing yang dijalankan. h2 dan h3 juga sama, ngikut tipe aktualnya.

2. Tambahkan class Ikan yang extend Hewan, override suara() dengan output "Blub - Blub!", lalu tambah objek Ikan pakai VMI. Apa yang terjadi?

```
Ikan.java (tambahan)
1      class Ikan extends Hewan {
2          @Override
3          public void suara() {
4              System.out.println("Blub - Blub!");
5          }
6      }
7
8      // Di main:
9      Hewan h4 = new Ikan();
10     h4.suara();
```

Output:

```
Blub - Blub!
```

Outputnya "Blub - Blub!" karena h4 tipe aktualnya Ikan. Ini nunjukin dynamic binding fleksibel, bisa tambah subclass baru tanpa ngubah kode lama.

8.4 Static Binding (Early Binding)

Static binding kebalikan dari dynamic binding. Method yang dipanggil udah ditentukan saat compile time. Ini berlaku buat method yang static, private, atau final karena nggak bisa di-override.

```
Kendaraan.java & Motor.java
1      public class Kendaraan {
2          public static void infoJenis() {
3              System.out.println("Kendaraan Umum");
4          }
5          public void klakson() {
6              System.out.println("Beep");
7          }
8      }
```

```

8      }
9
10     public class Motor extends Kendaraan {
11     public static void infoJenis() {
12         System.out.println("Motor");
13     }
14     @Override
15     public void klakson() {
16         System.out.println("Tin tin");
17     }
18 }

```

ContohStaticBinding.java

```

1     public class ContohStaticBinding {
2     public static void main(String[] args) {
3         Kendaraan k = new Motor();
4
5         Kendaraan.infoJenis(); // static binding
6         Motor.infoJenis(); // static binding
7
8         k.klakson(); // dynamic binding
9     }
10 }

```

Output:

```

Kendaraan Umum
Motor
Tin tin

```

Pertanyaan Analisa

1. Mengapa output method static dan method instance berbeda? Hubungkan dengan static vs dynamic binding!

Method static pake static binding, jadi ngikutin nama class yang manggil. Kendaraan.infoJenis() manggil versi Kendaraan, Motor.infoJenis() manggil versi Motor. Tapi klakson() pake dynamic binding, jadi ngikutin tipe objek aslinya (Motor), bukan tipe variabelnya (Kendaraan). Singkatnya: static binding liat tipe deklarasi, dynamic binding liat tipe aktual objek.

2. Ubah method klakson() di Kendaraan jadi final. Apa yang terjadi? Mengapa final method termasuk static binding?

Akan error saat compile karena method final nggak boleh di-override. Error: "klakson() in Motor cannot override klakson() in Kendaraan; overridden method is final". Final method termasuk static binding karena nggak bisa di-override, jadi compiler udah tahu pasti method mana yang kepanggil.

8.5 Upcasting dan 8.6 Downcasting

Upcasting = objek subclass dijadikan tipe superclass. Otomatis dan aman, tapi cuma bisa akses method superclass. **Downcasting** = kebalikannya, dari superclass ke subclass. Harus manual dan berisiko ClassCastException, makanya harus pake instanceof buat ngecek dulu.

Produk.java & Elektronik.java

```
1  public class Produk {
2      protected String nama;
3      protected double harga;
4
5      public Produk(String nama, double harga) {
6          this.nama = nama;
7          this.harga = harga;
8      }
9
10     public void tampilDetail() {
11         System.out.println("Produk: " + nama);
12         System.out.println("Harga: Rp " + harga);
13     }
14 }
15
16 public class Elektronik extends Produk {
17     private int garansiTahun;
18
19     public Elektronik(String nama, double harga, int garansiTahun) {
20         super(nama, harga);
21         this.garansiTahun = garansiTahun;
22     }
23
24     @Override
25     public void tampilDetail() {
26         super.tampilDetail();
```

```

27     System.out.println("Garansi: " + garansiTahun + " tahun");
28 }
29
30 public void cekGaransi() {
31     System.out.println("Garansi masih berlaku "
32 + garansiTahun + " tahun");
33 }
34 }

```

MainUpCasting.java

```

1     public class MainUpCasting {
2     public static void main(String[] args) {
3         Produk p = new Elektronik("Laptop Asus", 8500000, 2);
4         p.tampilDetail();
5         // p.cekGaransi(); // error: method tidak ada di Produk
6     }
7     }

```

Output:

```

Produk: Laptop Asus
Harga: Rp 8500000.0
Garansi: 2 tahun

```

p bertipe Produk tapi tampilDetail() yang jalan tetap milik Elektronik (dynamic binding). Tapi cekGaransi() nggak bisa dipanggil karena tipe referensinya Produk.

MainDownCasting.java

```

1     public class MainDownCasting {
2     public static void main(String[] args) {
3         Produk p = new Elektronik("iPhone 15", 15000000, 1);
4
5         if (p instanceof Elektronik) {
6             Elektronik e = (Elektronik) p;
7             e.tampilDetail();
8             e.cekGaransi(); // bisa diakses setelah downcasting
9         }
10    }
11    }

```

MainKasting.java

```

1     public class MainKasting {

```

```

2   public static void main(String[] args) {
3       Produk[] keranjang = {
4           new Elektronik("Headphone", 850000, 1),
5           new Produk("Tas Ransel", 250000),
6           new Elektronik("Smartwatch", 3200000, 2)
7       };
8
9       for (Produk item : keranjang) {
10          item.tampilDetail();
11
12          if (item instanceof Elektronik) {
13              Elektronik el = (Elektronik) item;
14              el.cekGaransi();
15          }
16          System.out.println("-----");
17      }
18  }
19  }

```

Output:

```

Produk: Headphone
Harga: Rp 850000.0
Garansi: 1 tahun
Garansi masih berlaku 1 tahun
-----
Produk: Tas Ransel
Harga: Rp 250000.0
-----
Produk: Smartwatch
Harga: Rp 3200000.0
Garansi: 2 tahun
Garansi masih berlaku 2 tahun
-----

```

Pertanyaan Analisa

1. Mengapa item bertipe Produk (seperti Tas Ransel) tidak menampilkan info garansi? Jelaskan peran instanceof!

Karena "Tas Ransel" dibuat pake new Produk(), bukan new Elektronik(). Dia nggak punya method cekGaransi(). Saat dicek instanceof, hasilnya false, jadi blok downcasting nggak dijalankan. Instanceof itu pengaman biar nggak sembarangan downcasting yang bisa bikin ClassCastException.

2. Ubah Produk p = new Elektronik(...) jadi Produk p = new Produk(...), lalu downcasting paksa tanpa instanceof: Elektronik e = (Elektronik) p;. Apa yang terjadi?

Akan muncul ClassCastException saat runtime. Objek p dibuat pake new Produk(), jadi tipe aslinya Produk, bukan Elektronik. Setiap Elektronik itu Produk, tapi nggak sebaliknya. Paksa downcasting pasti error. Ini bukti downcasting itu berisiko dan harus dicek pake instanceof dulu.

Tugas Praktikum: Sistem Kasir Restoran Cepat Saji

Buat program simulasi kasir restoran cepat saji pake polimorfisme. Ada 3 kategori menu: Makanan (harga dikali faktor ukuran), Minuman (tambah biaya ice), dan Paket (diskon 15% kalo beli 3 item atau lebih).

```
MenuItem.java
1  public class MenuItem {
2      protected String nama;
3      protected double harga;
4
5      public MenuItem(String nama, double harga) {
6          this.nama = nama;
7          this.harga = harga;
8      }
9
10     public double hitungTotal() {
11         return harga;
12     }
13
14     public void tampilStruk() {
15         System.out.println("Menu : " + nama);
16         System.out.println("Harga : Rp "
17             + String.format("%.0f", harga));
18         System.out.println("Total : Rp "
19             + String.format("%.0f", hitungTotal()));
20     }
21
22     public static double hitungPajak(double total) {
23         return total * 0.11;
24     }
25 }
```

Makanan.java

```
1  public class Makanan extends MenuItem {
2      private String ukuran; // S, M, L
3
4      public Makanan(String nama, double harga, String ukuran) {
5          super(nama, harga);
6          this.ukuran = ukuran;
7      }
8
9      @Override
10     public double hitungTotal() {
11         double faktor = 1.0;
12         switch (ukuran) {
13             case "S": faktor = 1.0; break;
14             case "M": faktor = 1.3; break;
15             case "L": faktor = 1.6; break;
16         }
17         return harga * faktor;
18     }
19
20     @Override
21     public void tampilStruk() {
22         super.tampilStruk();
23         System.out.println("Ukuran: " + ukuran);
24     }
25
26     public String getUkuran() { return ukuran; }
27 }
```

Minuman.java

```
1  public class Minuman extends MenuItem {
2      private boolean isIce;
3
4      public Minuman(String nama, double harga, boolean isIce) {
5          super(nama, harga);
6          this.isIce = isIce;
7      }
8  }
```

```

9      @Override
10     public double hitungTotal() {
11         return harga + (isIce ? 2000 : 0);
12     }
13
14     @Override
15     public void tampilStruk() {
16         super.tampilStruk();
17         System.out.println("Jenis : "
18             + (isIce ? "Ice" : "Hot"));
19     }
20
21     public boolean isIce() { return isIce; }
22 }

```

Paket.java

```

1     public class Paket extends MenuItem {
2         private int jumlahItem;
3
4         public Paket(String nama, double harga, int jumlahItem) {
5             super(nama, harga);
6             this.jumlahItem = jumlahItem;
7         }
8
9         @Override
10        public double hitungTotal() {
11            if (jumlahItem >= 3) {
12                return harga * 0.85; // diskon 15%
13            }
14            return harga;
15        }
16
17        @Override
18        public void tampilStruk() {
19            super.tampilStruk();
20            System.out.println("Jumlah Item: " + jumlahItem);
21            if (jumlahItem >= 3) {
22                System.out.println("Diskon : 15%");

```

```

23     }
24 }
25
26 public int getJumlahItem() { return jumlahItem; }
27 }

```

MainKasir.java

```

1  public class MainKasir {
2      public static void main(String[] args) {
3          MenuItem[] pesanan = {
4              new Makanan("Burger Ayam", 25000, "M"),
5              new Minuman("Teh Manis", 8000, true),
6              new Makanan("Kentang Goreng", 18000, "L"),
7              new Minuman("Kopi Susu", 15000, false),
8              new Paket("Paket Hemat", 50000, 3),
9              new Makanan("Nasi Goreng", 22000, "S"),
10             new Paket("Paket Keluarga", 85000, 4)
11         };
12
13         double subtotal = 0;
14         System.out.println("===== STRUK PESANAN =====");
15         System.out.println();
16
17         for (MenuItem item : pesanan) {
18             item.tampilStruk();
19
20             if (item instanceof Makanan) {
21                 Makanan m = (Makanan) item;
22                 System.out.println("[Info] Ukuran " + m.getUkuran()
23                     + " - Faktor harga: "
24                     + (m.getUkuran().equals("S") ? "1.0"
25                     : m.getUkuran().equals("M") ? "1.3" : "1.6"));
26             } else if (item instanceof Minuman) {
27                 Minuman mn = (Minuman) item;
28                 System.out.println("[Info] Tambahan ice: Rp "
29                     + (mn.isIce() ? "2.000" : "0"));
30             } else if (item instanceof Paket) {
31                 Paket pk = (Paket) item;

```

```

32     System.out.println("[Info] Jumlah item: "
33     + pk.getJumlahItem()
34     + (pk.getJumlahItem() >= 3
35     ? " (diskon 15%)" : ""));
36 }
37
38 subtotal += item.hitungTotal();
39 System.out.println("-----");
40 }
41
42 double pajak = MenuItem.hitungPajak(subtotal);
43 double totalBayar = subtotal + pajak;
44
45 System.out.println();
46 System.out.println("===== RINGKASAN =====");
47 System.out.println("Subtotal : Rp "
48 + String.format("%.0f", subtotal));
49 System.out.println("PPN 11% : Rp "
50 + String.format("%.0f", pajak));
51 System.out.println("TOTAL : Rp "
52 + String.format("%.0f", totalBayar));
53 System.out.println("=====");
54 }
55 }

```

Output:

===== STRUK PESANAN =====

Menu : Burger Ayam
Harga : Rp 25,000
Total : Rp 32,500
Ukuran: M
[Info] Ukuran M - Faktor harga: 1.3

Menu : Teh Manis
Harga : Rp 8,000
Total : Rp 10,000
Jenis : Ice
[Info] Tambahan ice: Rp 2.000

Menu : Kentang Goreng
Harga : Rp 18,000
Total : Rp 28,800
Ukuran: L
[Info] Ukuran L - Faktor harga: 1.6

Menu : Kopi Susu
Harga : Rp 15,000
Total : Rp 15,000
Jenis : Hot
[Info] Tambahan ice: Rp 0

Menu : Paket Hemat
Harga : Rp 50,000
Total : Rp 42,500
Jumlah Item: 3
Diskon : 15%
[Info] Jumlah item: 3 (diskon 15%)

Menu : Nasi Goreng
Harga : Rp 22,000
Total : Rp 22,000
Ukuran: S
[Info] Ukuran S - Faktor harga: 1.0

Menu : Paket Keluarga
Harga : Rp 85,000
Total : Rp 72,250
Jumlah Item: 4
Diskon : 15%
[Info] Jumlah item: 4 (diskon 15%)

===== RINGKASAN =====

Subtotal : Rp 223,050
PPN 11% : Rp 24,536
TOTAL : Rp 247,586

=====

Penjelasan:

- 1. Overriding:** Tiap subclass nulis ulang `hitungTotal()` dan `tampilStruk()`. Makanan kali faktor ukuran, Minuman tambah biaya ice, Paket kasih diskon 15%.
- 2. Dynamic Binding:** Semua objek disimpan di array `MenuItem` (upcasting). Pas dipanggil lewat loop, JVM otomatis jalanin method versi subclass sesuai tipe aktual objeknya.
- 3. Downcasting + instanceof:** Program downcasting pakai `instanceof` buat akses method khusus tiap kategori. `Instanceof` ngecek dulu biar aman dari `ClassCastException`.
- 4. Static Method:** `hitungPajak()` pake keyword `static`, bisa dipanggil langsung dari nama class. Ini contoh static binding.